Scientific Computing Laboratory

Lab 2

GMRES for Large Sparse Linear Systems

Experimental Report

8 October 2026

Abstract

This report reproduces all four tasks in the Lab 2 handout. Three prescribed sparse systems are solved with Gauss–Seidel, restarted GMRES, and ILU-preconditioned GMRES. A finite-difference boundary-value problem is solved on three grids with the two GMRES variants. We report true residuals, errors against known solutions, elapsed times, and solver-library cross-checks. At a common relative-residual target of 10^{-11}, Gauss–Seidel diverges for system 1.2, whereas both GMRES variants converge. In the tridiagonal problems, the specified incomplete factorization is effectively exact and preconditioned GMRES finishes in one Krylov step. The boundary-value errors decrease at approximately second order as the grid is refined.

1 Tasks and interpretation of the source

The handout prescribes three linear systems with $b = A\mathbf{1}$, hence the exact algebraic solution is $x^* = \mathbf{1}$. System 1.1 is a 300×300 tridiagonal matrix with diagonal 61.0001, subdiagonal 35, and superdiagonal 25. System 1.2 has size 100, diagonal $(1, 2, \ldots, 100)$, unit first off-diagonals, and two corner entries $a_{1,100} = a_{100,1} = 100$.

The ellipses in the printed system 1.3 do not define every entry formally. For reproducibility, we use the directly displayed pattern: size 100; diagonal 4; first lower and upper diagonals 2; second lower and upper diagonals 1; third lower diagonal 1; and the exceptional entries $a_{2,100} = 2$, $a_{100,2} = 3$. All unspecified entries are zero. This construction reproduces the displayed first, second, fourth, and last rows and has 593 nonzeros. The result is conditional on this explicit reading of the figure.

For the boundary-value problem, the handout prints d^2u/dt^2 although its independent variable, supplied difference equation, and lecture slide all use x. We therefore solve

$$u''(x) - (1+x)^2 u'(x) - e^{-x}u(x) = f(x), \quad 0 < x < 1, \qquad u(0) = 2, \quad u(1) = e - 1, \tag{1}$$

where

$$f(x) = [1 - (1+x)^2]e^x - \pi^2 \cos(\pi x) + \pi(1+x)^2 \sin(\pi x) - 1 - e^{-x}\cos(\pi x). \tag{2}$$

The known exact solution is $u(x) = e^x + \cos(\pi x)$. The handout requests $N = 50, 100, 200$; the slide's smaller grid examples are illustrative and are not substituted for these assigned grids.

2 Algorithms and experimental protocol

Gauss–Seidel uses $A = L + U$ with L including the diagonal and iterates $Lx^{(k+1)} = b - Ux^{(k)}$. GMRES builds a Krylov basis by modified Gram–Schmidt with a second orthogonalization pass. At each inner step it solves the small least-squares problem and records the actual residual of the reconstructed iterate. Restart length is 30. Right preconditioning uses an incomplete LU factorization $M \approx A$, with $z_j = M^{-1}v_j$ and Arnoldi applied to Az_j. The reported ILU implementation is SciPy SuperLU's threshold ILU (drop tolerance 10^{-4}, fill factor 10, natural ordering), rather than a separately coded ILU(0). SciPy's reference `gmres` applies its `M` argument as a left preconditioner; we use it to check the final solution and true residual, not to require identical intermediate iterates.

All solvers start from zero. Convergence requires $\|b - Ax_m\|_2 \le 10^{-11}\|b\|_2$, evaluated using the *original*, unpreconditioned system. The iteration cap is 10,000. Gauss–Seidel is also stopped and labeled divergent if its residual exceeds 10^{12} times its initial value or becomes non-finite. This safeguard does not count as convergence. We report $\|b - Ax_m\|_2$ and, for systems 1.1–1.3, $\|x^* - x_m\|_2$.

Each method/problem combination is run five times. The table gives the median wall time, including matrix splitting or ILU setup and per-iteration true-residual/error sampling. The plotted history is the median-time run, and its horizontal axis measures solve time after setup. Timings are short, machine-dependent Python measurements; the iteration and error comparisons are more stable than sub-millisecond timing ratios. NumPy 2.5.3, SciPy 1.18.1, Python 3.13.7, arm64 are used.

3 Three prescribed sparse systems

Table 1: Solver results for the three algebraic systems. Time is the median of five runs and includes setup. The error column uses the known solution $\mathbf{1}$.

Case	Method	Iter.	$\|b - Ax\|_2$	$\|x - \mathbf{1}\|_2$	Time (ms)	Outcome
1.1	Gauss–Seidel	326	2.04×10^{-8}	1.96×10^{-8}	41.682	converged
1.1	GMRES	192	2.05×10^{-8}	1.43×10^{-8}	14.353	converged
1.1	ILU–GMRES	1	2.06×10^{-13}	6.44×10^{-15}	0.122	converged
1.2	Gauss–Seidel	7	1.04×10^{16}	1.57×10^{14}	0.898	diverged
1.2	GMRES	91	5.83×10^{-9}	2.77×10^{-9}	6.343	converged
1.2	ILU–GMRES	2	8.28×10^{-12}	1.95×10^{-12}	0.110	converged
1.3	Gauss–Seidel	119	1.06×10^{-9}	2.68×10^{-9}	14.207	converged
1.3	GMRES	61	1.07×10^{-9}	8.23×10^{-10}	4.373	converged
1.3	ILU–GMRES	3	3.88×10^{-14}	6.12×10^{-15}	0.130	converged

The tridiagonal system 1.1 converges under all three methods. The preconditioner preserves its tridiagonal elimination pattern, so the factorization is effectively exact and one Krylov step suffices. For system 1.2, Gauss–Seidel exhibits rapid growth instead of convergence. Its first row is far from diagonally dominant because of the 100 corner coefficient; this explains why the standard sufficient convergence condition does not apply, while the measured divergence establishes the outcome for this particular ordering. Both GMRES variants converge. For system 1.3, all three converge, with ILU–GMRES requiring the fewest Krylov steps. These observations concern the specified matrices and solver settings, not a universal ranking.

4 Finite-difference boundary-value problem

For $x_i = ih$, $h = 1/N$, centered differences give

$$\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} - (1 + x_i)^2 \frac{u_{i+1} - u_{i-1}}{2h} - e^{-x_i} u_i = f(x_i), \tag{3}$$

$$[2 + h(1 + x_i)^2]u_{i-1} - [4 + 2h^2 e^{-x_i}]u_i + [2 - h(1 + x_i)^2]u_{i+1} = 2h^2 f(x_i). \tag{4}$$

Boundary terms $u_0 = 2$ and $u_N = e - 1$ are moved into the right-hand side of the $(N-1) \times (N-1)$ system. For the interior solution u_h, we compute

$$E_\infty = \max_{1 \le i \le N-1} |u_i - u(x_i)|, \qquad E_{2,h} = \left[h \sum_{i=1}^{N-1} (u_i - u(x_i))^2 \right]^{1/2}. \tag{5}$$

Table 2: Boundary-value results. Each grid uses $N - 1$ interior unknowns; time is median total time including setup.

N	Method	Iter.	$\|F - AU\|_2$	E_∞	$E_{2,h}$	Time (ms)
50	GMRES	298	5.04×10^{-11}	4.97×10^{-4}	3.29×10^{-4}	19.465
50	ILU–GMRES	1	4.04×10^{-15}	4.97×10^{-4}	3.29×10^{-4}	0.074
100	GMRES	915	5.25×10^{-11}	1.24×10^{-4}	8.22×10^{-5}	63.330
100	ILU–GMRES	1	5.70×10^{-15}	1.24×10^{-4}	8.22×10^{-5}	0.093
200	GMRES	3207	5.24×10^{-11}	3.11×10^{-5}	2.06×10^{-5}	237.684
200	ILU–GMRES	1	9.79×10^{-15}	3.11×10^{-5}	2.06×10^{-5}	0.168

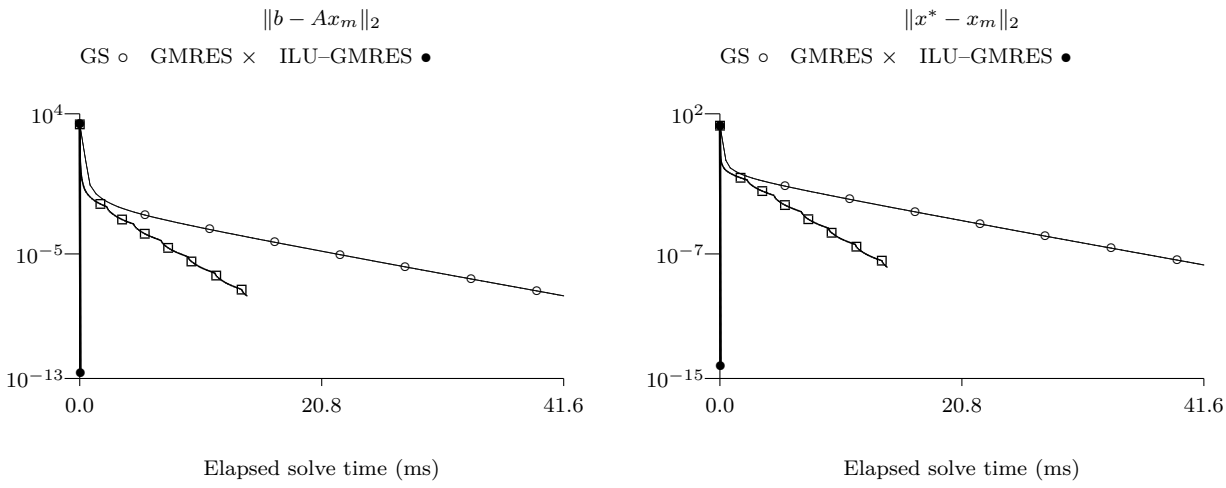


Figure 1: Experiment 1.1: residual and solution error against solver time. Only plotted points are thinned; all recorded iterations remain in the JSON data. All three methods use the same relative-residual target.

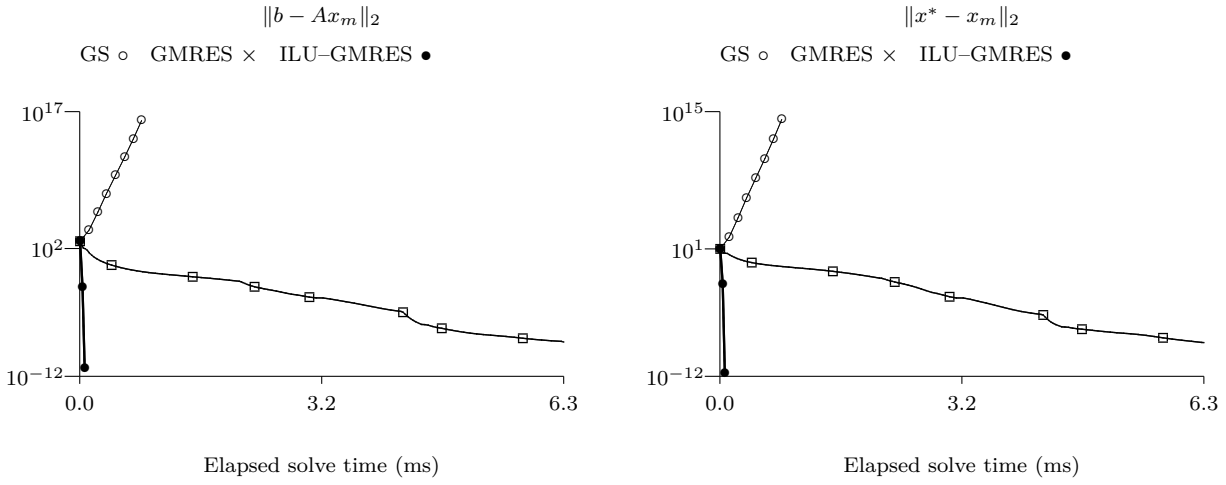


Figure 2: Experiment 1.2: residual and solution error against solver time. Only plotted points are thinned; all recorded iterations remain in the JSON data. The Gauss–Seidel curve stops at the divergence safeguard.

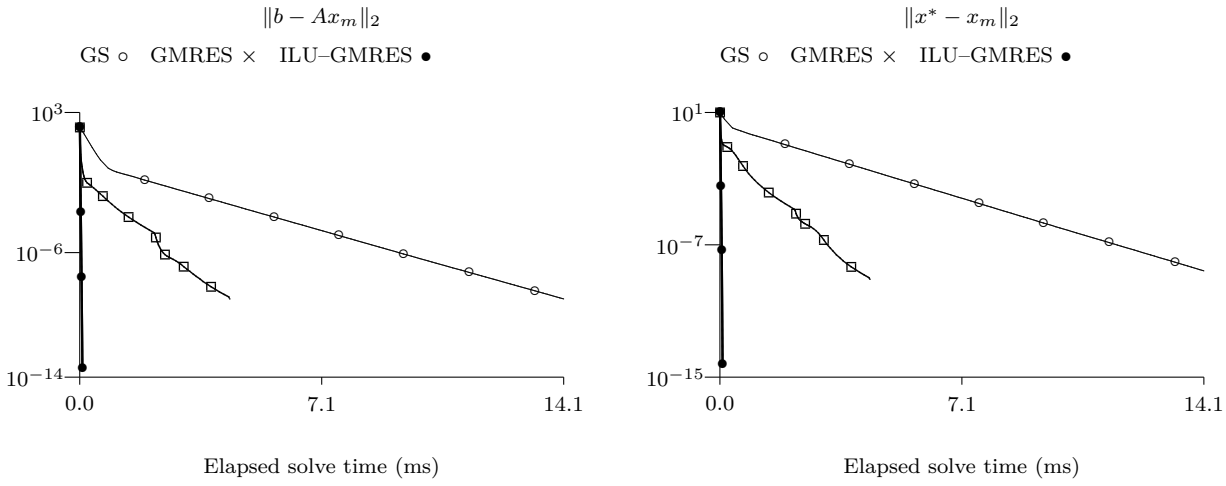


Figure 3: Experiment 1.3: residual and solution error against solver time. Only plotted points are thinned; all recorded iterations remain in the JSON data. All three methods use the same relative-residual target.

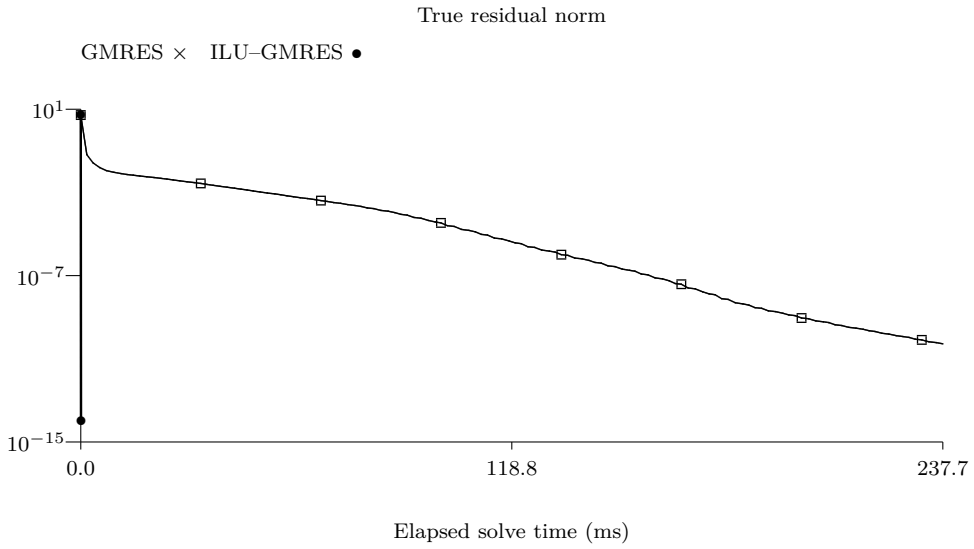


Figure 4: True residual history for the finest boundary-value grid ($N = 200$). The GMRES curve is decimated for display only.

Table 3: Observed grid-convergence orders.

Method	E_∞: 50–100	E_∞: 100–200	$E_{2,h}$: 50–100	$E_{2,h}$: 100–200
GMRES	2.000	2.000	2.000	2.000
ILU–GMRES	2.000	2.000	2.000	2.000

The measured convergence order between grids N and $2N$ is $p = \log_2(E_N/E_{2N})$.

Both error measures approach the expected second-order behavior of centered finite differences. The unpreconditioned GMRES iteration count rises sharply on finer grids. With this tridiagonal operator and the chosen ILU settings, the factorization reproduces the direct solve to roundoff; ILU–GMRES therefore uses one Krylov step at every grid size. The remaining error is primarily the discretization error, not a failure of the linear solver. A looser linear-solver tolerance distorted the finest-grid error, so the reported run uses 10^{-11} throughout.

5 Independent checks and limitations

The code checks selected displayed matrix entries, the boundary values, and the nonzero count. A sparse direct solve provides an independent algebraic reference for every case. Independently called `scipy.sparse.linalg.gmres` returns success for every custom GMRES run, and their solutions agree to within 6.142×10^{-9} in Euclidean norm; the largest difference occurs in the ill-conditioned finest-grid solve. Direct-solve residuals are at most 5.581×10^{-14}. The two boundary-value error measures and their orders are computed against the analytic solution, not against the direct numerical solve.

The handout asks students to compare AI-assisted code with library functions. Here the custom Arnoldi/GMRES implementation is cross-checked with SciPy's GMRES and a sparse direct solve; neither library result is treated as an exact continuum solution. Timing includes instrumentation and is not a low-level performance benchmark. The 1.3 matrix interpretation, ILU parameters, stopping rule, and divergence rule are fully stated so that the results can be reproduced or revised if the instructor supplies a more explicit matrix.

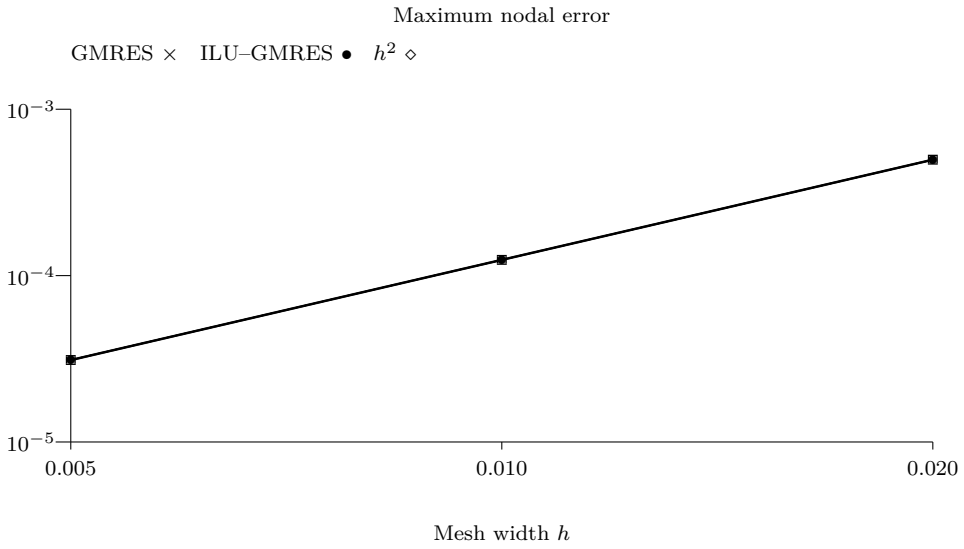


Figure 5: Observed maximum nodal error versus mesh width. The h^2 line is a slope-two reference, not a fitted model.

6 Conclusion

All assigned systems and grid sizes were completed. GMRES handled the case where Gauss–Seidel diverged. ILU preconditioning greatly reduced Krylov iterations for these matrices, reaching one step when the factorization was effectively exact. The finite-difference solution showed approximately second-order convergence under grid refinement. True residual checks, known-solution errors, sparse direct solves, and SciPy GMRES cross-checks support the numerical results.

A Reproduction

From the `experiment` directory, run `../../.venv/bin/python run_experiment.py`, followed by `../../.venv/bin/python build_report.py`. The first command writes `results/results.json`; the second regenerates this standalone `main.tex`, with all plotted data embedded. Compile the source in the Codex LaTeX editor. The provided handout and lecture slides remain in the parent experiment directory.